07

Estratégias de Cache e Redis

Estratégias de Cache e Redis

Como acelerar uma API simples sem devolver dados antigos.

Roteiro da aula

- Problema real na API
- Cache em APIs
- Cache com Laravel
- Cache por recurso
- Chaves e tags por recurso
- Invalidação inteligente
- Redis no backend Laravel
- Eventos entre componentes
- Rate limiting com Redis

Problema real na API

Definição: API de tarefas com endpoints de leitura.
Quando usar: Leituras repetidas com os mesmos filtros.
Exemplo prático: GET /api/tasks?status=open.

Exemplo prático de código

```
GET /api/tasks?status=open
Accept: application/json
```

Cache em APIs

Definição: Cache reutiliza respostas por um período controlado.
Quando usar: Leituras aceitam pequena defasagem.
Exemplo prático: GET /api/tasks em cache por 30 segundos.

```
Cache-Control: public, max-age=30
X-Cache: HIT
```

Cache com Laravel

Definição: Cache::remember retorna o valor armazenado ou executa a consulta.
Quando usar: A leitura possui uma chave clara e um tempo de validade definido.
Exemplo prático: A lista de tarefas abertas usa a chave tasks:list:open por 30 segundos.

Exemplo prático de código

```
$tasks = Cache::remember(
    "tasks:list:{$status}",
    now()->addSeconds(30),
    fn () => Task::query()
        ->where('status', $status)
        ->get(),
);
```

Cache por recurso

Definição: Listas e detalhes usam chaves independentes
Quando usar: Uma mudança afeta somente parte da API.
Exemplo prático: tasks:list:open e tasks:42.

```
SETEX tasks:list:open 30 "[...]"
SETEX tasks:42 300 "{...}"
```

Chaves e tags por recurso

Definição: Tags agrupam chaves que pertencem ao mesmo recurso.
Quando usar: Uma alteração precisa invalidar listas e detalhes relacionados.
Exemplo prático: A tag tasks remove tasks:list:open e tasks:42 em uma única operação.

Exemplo prático de código

```
Cache::tags(['tasks'])
    ->remember(
        "list:{$status}",
        30,
        fn () => Task::query()
            ->where('status', $status)
            ->get(),
    );
```

Invalidação inteligente

Definição: Invalidação remove o cache após uma escrita.
Quando usar: POST, PUT ou DELETE mudam respostas.
Exemplo prático: PUT /api/tasks/42 limpa duas chaves.

```
Cache::tags(['tasks'])->flush();
Cache::forget('tasks:42');
```

Redis no backend Laravel

Definição: Redis compartilha cache entre instâncias da API.
Quando usar: A aplicação roda em mais de um servidor.
Exemplo prático: Todas as instâncias leem tasks:list:open.

```
GET tasks:list:open
TTL tasks:list:open
```

Eventos entre componentes

Definição: Pub/Sub distribui eventos entre componentes.
Quando usar: Serviços precisam reagir sem bloquear a API.
Exemplo prático: A API publica task.updated.

```
Redis::publish(
    'task.updated',
    json_encode(['id' => 42]),
);
```

Rate limiting com Redis

Definição: Rate limiting restringe requisições por período.
Quando usar: Endpoints públicos precisam limitar abuso.
Exemplo prático: 60 requisições por minuto por token.

```
Route::middleware('throttle:60,1')
    ->post('/tasks', StoreTaskController::class);
```

Cache com Observers

Comando para vincular observer na model

```
#[ObservedBy(\App\Observers\TaskObserver::class)]
```

Para criar um observer:

```
php artisan make:observer
```

Projeto do módulo

Definição: O projeto aplica cache na API de tarefas.
Quando usar: Reunir leitura, invalidação e Redis.
Exemplo prático: Cachear GET e invalidar após escrita.

```
GET /api/tasks
POST /api/tasks
PUT /api/tasks/42
DELETE /api/tasks/42
```

Checklist final

Definição: O checklist valida a estratégia completa.
Quando usar: Revisar a API antes de concluir o módulo.
Exemplo prático: GET reutiliza; PUT invalida o cache.

```
php artisan test
redis-cli MONITOR
curl /api/tasks
```